ways to configure CORS requests. The `handleCors` method is updated to accept an `options` object, and returns a new middleware object. The `corsOptions` variable is passed to the `handleCors` method, and you receive a fully configured middleware.

Now that the sample code framework is in place, you can focus on tips for setting each individual header in the CORS pipeline. But before doing that, let's take a few moments to think about your own server configuration.

6.2 Before you begin

Before writing a single line of code, take a step back and think about what you're trying to achieve by adding CORS support to your server. Here are a few questions you should consider:

- Why are you adding support for cross-origin requests?
- Are you adding CORS support to a new service or an existing server?
- Which clients should have access to the site?
- What devices/browsers will they be accessing the site from?
- Which HTTP methods and headers will your server support?
- Should the API support user-specific data? If so, will cookies be used to authenticate the user?

The answers will guide you through the rest of this chapter. For example, an API that needs Internet Explorer 8 or Internet Explorer 9 support will need to be built differently from an API that only needs to be supported on mobile devices. An API that is open to the public will have different needs than an API built for internal use. Here are some common server configurations that may benefit from CORS support:

- *Providing a public API that users can access from JavaScript.* Most Google APIs support CORS, including the popular Drive API and YouTube API.
- *Unifying various properties behind a single API.* If you own a mobile, tablet, and website, and each is hosted under a different origin, they can all talk to the same underlying API using CORS.

By having an understanding of what you're trying to achieve with CORS, you can make better choices on how to configure CORS for your server. Now that you have a sense of how your own CORS implementation should behave, let's turn our attention to the most important CORS header: Access-Control-Allow-Origin.

6.3 Setting the Access-Control-Allow-Origin header

The Access-Control-Allow-Origin header indicates which origins can access a resource via a cross-origin request. This header can either use the `*` value to give access to everyone, or it can set a specific origin value to give access to a specific client. Table 6.1 gives an overview of the valid values for the Access-Control-Allow-Origin header. The rest of this chapter delves deeper into these values, and explores different use cases.